

## Node Types

|             |                       |
|-------------|-----------------------|
| Cache       | Caching station       |
| ClassSwitch | Class switching       |
| Delay       | Infinite server       |
| Fork        | Forks jobs            |
| Join        | Joins jobs            |
| Logger      | Trace logging         |
| Place       | Petri net place       |
| Queue       | Queueing station      |
| Router      | Routing node          |
| Sink        | Job departures (open) |
| Source      | Job arrivals (open)   |
| Transition  | Petri net transition  |

## Job Classes

ClosedClass(model, 'name', N, refStat)  
OpenClass(model, 'name')  
SelfLoopingClass(model, 'name', N, ref)  
DisabledClass(model, 'name')  
OpenSignal, ClosedSignal (catastrophes)

## Distributions

|                                         |                   |
|-----------------------------------------|-------------------|
| APH( $\alpha$ , T)                      | Acyclic PH        |
| Cox2( $\mu$ , $\phi$ )                  | Two-phase Coxian  |
| Coxian( $\mu$ , $\phi$ )                | Coxian            |
| Det( $v$ )                              | Deterministic     |
| Disabled                                | None              |
| Erlang( $\lambda$ , k)                  | Erlang-k          |
| Exp( $\lambda$ )                        | Exponential       |
| Gamma( $\alpha$ , $\beta$ )             | Gamma             |
| HyperExp(p, $\lambda_1$ , $\lambda_2$ ) | Hyper-exp         |
| Immediate                               | Zero service time |
| Lognormal( $\mu$ , $\sigma$ )           | Log-normal        |
| Normal( $\mu$ , $\sigma$ )              | Normal            |
| Pareto( $\alpha$ , k)                   | Pareto            |
| PH( $\alpha$ , T)                       | Phase-type        |
| Uniform(a, b)                           | Uniform           |
| Weibull( $\alpha$ , $\beta$ )           | Weibull           |

### Discrete

|                       |                |
|-----------------------|----------------|
| Bernoulli(p)          | Bernoulli      |
| Binomial(p, n)        | Binomial       |
| DiscreteUniform(a, b) | Discr. uniform |
| Geometric(p)          | Geometric      |
| Poisson( $\lambda$ )  | Poisson        |
| Zipf(s, n)            | Popularity     |

### Correlated / trace-driven

|                                                              |                       |
|--------------------------------------------------------------|-----------------------|
| BMAP (D0...Dk)                                               | Batch MAP             |
| NHPP (bp, $\lambda$ , cyclic)                                | Non-homog. Poisson    |
| DMAP (D0, D1)                                                | Discrete MAP          |
| EmpiricalCDF (F)                                             | Empirical CDF         |
| MAP (D0, D1)                                                 | MAP                   |
| MarkedMAP                                                    | Marked MAP            |
| ME( $\alpha$ , T)                                            | Matrix-exp.           |
| CME (mean, order)                                            | Concentr. matrix-exp. |
| MMPP2( $\lambda_1$ , $\lambda_2$ , $\sigma_1$ , $\sigma_2$ ) | MMPP(2)               |
| RAP (H0, H1)                                                 | Rational AP           |
| RePlayer (trace)                                             | Trace replay          |

### Marked / time-inhomogeneous

|                                            |                               |
|--------------------------------------------|-------------------------------|
| MAPt (bp, D0, D1, cyclic)                  | Time-varying MAP              |
| PHt (bp, $\alpha$ , S, cyclic)             | Time-varying PH               |
| MPH ( $\alpha$ , S, { $s_k$ })             | Marked PH                     |
| MMAPt (bp, D0, D1k, cyclic)                | Marked time-varying MAP       |
| MPHT (bp, $\alpha$ , S, { $s_k$ }, cyclic) | Marked time-varying PH        |
| BMMAPt (bp, D0, D1k, cyclic)               | Batch marked time-varying MAP |

A mark selects the CLASS of the arriving job (setMarkedArrival); a BMMAPt epoch releases a whole batch of that class. As a service process the mark is discarded, and a

BMAP or BMMAPt service is a bulk server. Declared by LDES; MAPt/PHT also by FLD

method kp.

## Scheduling Strategies

|         |                             |
|---------|-----------------------------|
| DPS     | Discriminatory PS           |
| EDD     | Earliest Due Date           |
| EDF     | Earliest Deadline First     |
| EXT     | External (source)           |
| FB      | Feedback (LAS)              |
| FCFS    | First-Come First-Served     |
| FCFSPI  | FCFS Preempt. Resume-idle   |
| FCFSPR  | FCFS Preemptive Resume      |
| FSP     | Fair Sojourn Protocol       |
| GPS     | Generalized PS              |
| HOL     | Head-of-Line Priority       |
| INF     | Infinite Servers            |
| LCFS    | Last-Come First-Served      |
| LCFSPI  | LCFS Preempt. Resume-idle   |
| LCFSPR  | LCFS Preemptive Resume      |
| LEPT    | Longest Expected PT         |
| LJF     | Longest Job First           |
| LPS     | Limited PS                  |
| LRPT    | Longest Remaining PT        |
| OI      | Order Independent           |
| PAS     | Pass and Swap               |
| POLLING | Polling (switchover)        |
| PS      | Processor Sharing           |
| PSJF    | Preemptive SJF              |
| SEPT    | Shortest Expected PT        |
| SETF    | Shortest Elapsed Time First |
| SIRO    | Random                      |
| SJF     | Shortest Job First          |
| SRPT    | Shortest Remaining PT       |

## Solvers

|      |                              |
|------|------------------------------|
| AUTO | Auto-select best             |
| BA   | Bound analysis (closed)      |
| CTMC | CTMC (exact, small)          |
| ENV  | Random environments          |
| FLD  | Fluid/mean-field ODEs        |
| LDES | Discrete Event Sim           |
| LN   | Layered networks             |
| MAM  | Matrix Analytic Methods      |
| MVA  | Mean Value Analysis          |
| NC   | Normalizing Constant Methods |
| SSA  | Stochastic simulation        |

## Wrappers

|      |                          |
|------|--------------------------|
| JMT  | JMT simulator            |
| LQNS | LQNS interface           |
| QNS  | LQNS product-form solver |

## Solver Options

```
MVA(model, 'method', 'exact')
JMT(model, 'seed', 123,
'samples', 10000)
SSA(model, 'samples', 5000,
'method', 'para')
LDES(model, MVA(model)) % warm start
```

## Output Metrics

|        |                                      |
|--------|--------------------------------------|
| ArvR   | Arrival rate                         |
| QLen   | Mean queue length                    |
| ResidT | Residence time (total across visits) |
| RespT  | Response time (per visit)            |
| Tput   | Throughput                           |

## Util

|           |                                          |
|-----------|------------------------------------------|
| QLenVar   | Queue-length variance                    |
| QLenSCV   | Queue-length squared coeff. of variation |
| QLenSkew  | Queue-length skewness                    |
| RespTVar  | Response-time variance (FCFS only)       |
| RespTSCV  | Response-time SCV (FCFS only)            |
| RespTSkew | Response-time skewness (FCFS only)       |

## Moments (product form)

```
solver.getMomentTable()
% per class; order 2 by default
solver.getMomentTable(3)
% l=means 2+=var 3+=skew
solver.getMomentChainTable(3)
% per chain
solver.getMomentStationTable(3)
% per station total
solver.getMomentStationTable(3, 'lin')
% approx., for large N
```

order is a set: scalar  $k$  = up to  $k$ ;  
a vector, e.g. [2, 3], = exactly those.  
RespT\* moments are FCFS-only  
(NaN elsewhere); exact methods cost  
 $\prod_r (N_r + 1)$ , use 'lin' beyond that.

## Analysis Methods

```
solver.avgTable() % alias: aT
solver.avgQLen(), avgUtil()
solver.avgRespT(), avgTput()
solver.avgSysRespT()
solver.avgSysTput()
solver.avgQLenChain()
```

## Method Aliases

|      |                      |
|------|----------------------|
| aCT  | getAvgChainTable     |
| aNT  | getAvgNodeTable      |
| aNCT | getAvgNodeChainTable |
| aST  | getAvgSysTable       |
| aT   | getAvgTable          |

## Routing

```
model.link(Network.serialRouting(
n1,n2,n3));
n1.setProbRouting(class, n2, 0.7);
n1.setRouting(class,
RoutingStrategy.RROBIN);
```

## Multi-Server & Finite Capacity

```
queue.setNumServers(k);
queue.setCapacity(bufferSize);
```

## Class Switching

```
cs = ClassSwitch(model, 'CS');
P = [0.3 0.7; 0.5 0.5];
cs.setClassSwitchingMatrix(P);
```

## Fork-join Networks

```
fork = Fork(model, 'Fork');
join = Join(model, 'Join', fork);
```

## Layered Networks

```
lqn = LayeredNetwork('name');
P = Processor(lqn, 'P', 1,
SchedStrategy.PS);
T = Task(lqn, 'T', 1,
SchedStrategy.REF).on(P);
E = Entry(lqn, 'E').on(T);
A = Activity(lqn, 'A').on(T)
.boundTo(E).setHostDemand(Exp(1));
```

## Caching

```
cache = Cache(model, 'Cache',
nItems, cap, ReplacementStrategy.LRU);
cache.setRead(class, Zipf(1.4, nItems));
% cache results
MVA(model).getAvgCacheTable() % hit/miss
MVA(model).getAvgItemTable() % per-item
```

### ReplacementStrategy

|       |                    |
|-------|--------------------|
| CLIMB | Climb              |
| FIFO  | First-in first-out |

## HLRU

|       |                     |
|-------|---------------------|
| h-LRU | Least recently used |
| LRU   | Least recently used |
| QLRU  | q-LRU               |
| RR    | Random replacement  |
| SFIFO | Static FIFO         |

## Model Import/Export

```
model.view() % Open in JMT
linemodel_save(model, 'model.json')
model = linemodel_load('model.json')
```

## Routing Strategies

|          |                       |
|----------|-----------------------|
| JSQ      | Join shortest queue   |
| KCHOICES | Power of k choices    |
| PROB     | Probabilistic routing |
| RAND     | Random selection      |
| RROBIN   | Round-robin           |
| WROBIN   | Weighted round-robin  |

## Stochastic Petri Nets

```
p = Place(model, 'P1');
t = Transition(model, 'T1');
m = t.addMode('Model');
t.setNumberOfServers(m, Inf);
t.setDistribution(m, Exp(4));
t.setEnablingConditions(m, class, p, 1);
t.setInhibitingConditions(m, class, p, 3);
t.setFiringOutcome(m, class, sink, 1);
```

## Markov Chains (solved directly)

```
ctmc = MarkovProcess([-0.5 0.5; 0.2 -0.2]);
pi = SolverCTMC(ctmc).getProbSys();
dtmc = MarkovChain([0.9 0.1; 0.4 0.6]);
piD = SolverCTMC(dtmc).getProbSys();
piT = ctmc.transient([1; 10]);
jump = ctmc.toEmbedded();
```

## Random Environments

```
env = Environment('R');
env.addStage('Fast', 'operational', fastModel);
env.addStage('Slow', 'degraded', slowModel);
env.addTransition('Fast', 'Slow', Exp(0.5));
env.addTransition('Slow', 'Fast', Exp(1.0));
ENV(env, @(m) FLD(m)).getAvgTable()
```

## Finite Capacity Regions

```
fcr = model.addRegion({queue1, queue2});
fcr.setGlobalMaxJobs(10);
fcr.setClassMaxJobs(class1, 5);
fcr.setDropRule(class1, DropStrategy.Drop);
```

## Load-Dependent Service

```
queue.setLoadDependence(min(1:N, c));
queue.setClassDependence(@(ni) min(sum(ni), c), c);
```

## Impatience & Retrials

```
queue.setPatience(class,
ImpatienceType.RENEGING, Exp(0.1));
queue.setBalking(class, strategy, thresholds);
queue.setRetrial(class, Exp(2), maxAttempts);
```

## State Probabilities

```
solver.getProb(node) % marginal
solver.getProbAggr(node) % aggregate
solver.getProbSys() % joint
SolverNC(m).getProbSysMarg(n) % totals
solver.getGenerator() % CTMC infin.
solver.getStateSpace()
```

## Transient & Distributions

```
solver.getTranAvg() % transient means
solver.getCdfRespT() % response-time CDF
model.setReward('Cost', @(state) ...);
solver.getAvgReward()
```

## Symbolic Analysis

```
options.config.symbolic = 'sage'; % needs line-sage-rest
[pi,num,den,sp] = solver.getSymbolicSolution();
[S,SS] = solver.getSensitivity(p, r, 'symbolic');
J = SolverFLD(model,o).getJacobian(o);
SAGE.ctmcMeasures(Q,syms,W); SAGE.ctmcPassage(...)
```